

CSCE 636 DEEP LEARNING · MODULE 09 · LECTURE NOTES

Transfer and Representation Learning

Reusing what a model already learned.

MODULE OUTCOMES

- Explain why pretrained features transfer.
- Choose between feature extraction and fine-tuning.
- Explain self-supervised pretraining and its objectives.
- Apply parameter-efficient fine-tuning.
- State when transfer fails.

1 Why transfer works

EARLY FEATURES ARE GENERAL; LATE FEATURES ARE SPECIFIC

Module 05 §2 found that early layers learn edge and colour detectors, middle layers textures and parts, and late layers whole objects — a hierarchy that emerged rather than being designed.

Edges are edges regardless of the task. So the early layers of a model trained on *anything* visual are useful for *everything* visual, and there is no reason to learn them again from a thousand examples when they were learned once from a million.

Late layers encode the specific categories the model was trained on, and those are exactly what you discard and replace with a head for the new task.

So transfer is: keep the general part, replace the specific part, and optionally adjust the middle. Transferability declines smoothly with depth — which has been measured directly, layer by layer — **and the measurement tells you how much to freeze** rather than requiring a guess.

Strategy	What changes	Use when
Feature extraction	Only a new head; the backbone is frozen entirely.	Very little data (hundreds of examples), or a deployment budget that cannot afford to store a full model per task. Fast and nearly impossible to get wrong.
Fine-tune the last layers	The head plus the top block or two.	Moderate data and a similar domain. A sensible default.
Full fine-tuning	Every parameter, at a much lower learning rate.	Plenty of data, or a genuine domain shift that the early features do not cover.

Strategy	What changes	Use when
LoRA and adapters	Only small added matrices; the base weights stay frozen.	Large models, many tasks, or limited memory — §3.
Prompting / in-context learning	Nothing. No gradient step at all.	Very large models with a handful of examples, where the adaptation happens in the forward pass.

2 Self-supervised pretraining

THE PRETEXT TASK CREATES LABELS FROM STRUCTURE

Supervised pretraining needs labels, and labels are the expensive and rate-limiting resource (CSCE 633 Module 11 §4) — ImageNet's fourteen million labels took years and considerable money.

Self-supervision invents a task whose labels come from the data itself: predict the next token in a sentence; reconstruct a masked patch of an image; decide whether two crops came from the same photograph. **No annotation is required at any point.**

Solving the pretext task requires learning genuinely useful structure — you cannot predict the next word without representing syntax, semantics, and a good deal of world knowledge — **and that representation transfers**, even though nobody cares about the pretext task's output.

This is why language models scale as they do. The internet is a labelled dataset if the label is 'the next word' — which removed, at a stroke, the data constraint that had bounded every previous approach. **The bottleneck moved from labelling to compute**, and compute was something that could be bought.

Objective	Where the signal comes from	Domain
Next-token prediction	The next element of the sequence, which the data already contains.	Text, and the dominant objective by a wide margin. Also audio and increasingly video.
Masked prediction	Hide part of the input and reconstruct it from the rest.	Text (BERT) and images (masked autoencoders), where masking a large fraction of patches turns out to work remarkably well.
Contrastive	Two augmented crops of one image should agree; crops of different images should not.	Images — SimCLR, MoCo. Requires careful augmentation design, since the augmentations define what the model is told to be invariant to (Module 04 §4).
Multimodal contrastive	An image and its caption should agree; mismatched pairs should not.	CLIP — and it changed a great deal. See below.
Denoising	Reconstruct a clean input from a corrupted one.	Generalises widely, and is the direct link to diffusion models (Module 10 §3).

CLIP's contribution was larger than a better representation. By aligning images and text in one embedding space it produced a model that could classify into arbitrary categories specified in natural language, with no task-specific training at all — **zero-shot classification, which was not expected to**

work. It also became the text-conditioning mechanism for image generation (Module 10), which is how a single pretraining objective ended up underneath most of generative vision.

3 Parameter-efficient fine-tuning

OBSERVATION: the weight UPDATE during fine-tuning has low intrinsic rank -- adapting to a new task does not require moving in many directions.

SO instead of updating W ($d \times k$), learn
 $W' = W + BA$ B is $d \times r$, A is $r \times k$, $r \ll d$

PARAMETERS TRAINED: $r(d+k)$ instead of dk
 $d = k = 4096$, $r = 8 \rightarrow 65,536$ rather than $16,777,216$
 a 256x reduction

PROPERTIES

- * W is FROZEN, so pretrained knowledge cannot be destroyed
- * BA MERGES into W after training -- ZERO inference cost
- * many task adapters share one base model: the real deployment win
- * initialise B to ZERO so the adapted model starts EXACTLY equal to the base model

Quality is typically within a point or two of full fine-tuning, at a small fraction of the memory.

The zero-initialisation of B is the detail people miss. It makes the product BA exactly zero at the start, so training begins from the pretrained model rather than from a perturbed version of it — which matters because the adapter's randomly initialised weights would otherwise inject noise into a model that was already good. **It is the same idea as initialising residual branches near zero** (Module 03 §3).

4 When transfer fails

TRANSFER FAILS WHEN THE DOMAINS GENUINELY DIFFER

Natural images to medical imaging transfers poorly. The low-level statistics are different, the relevant features are different, and **careful studies have found that much of the apparent benefit comes from the scale of the pretrained model and from better initialisation statistics rather than from the transferred features themselves** — a much weaker claim than 'ImageNet features help'.

Natural images to satellite imagery, to microscopy, to infrared: different spatial scales, different invariances (a satellite image has no canonical 'up' for objects), different noise characteristics.

And negative transfer is real. A pretrained model can perform worse than random initialisation when the source domain's biases actively mislead — the model arrives committed to features that are wrong for the target and must unlearn them.

So measure it. Train from scratch as a baseline whenever transfer is used, on the same data and the same split. This is CSCE 633 Module 01 §4's baseline rule applied to transfer, **and it is routinely skipped** because transfer is assumed to help — which it usually does, and not always, and the difference is one extra training run.

- **Use a much lower learning rate** — typically 10 to 100× below what you would use training from scratch. **Too high a rate destroys the pretrained features in the first few steps**, which is the classic fine-tuning failure and looks like 'transfer did not help'.
- **Warm up, and consider freezing the backbone for the first epoch** while the randomly initialised head stabilises. **A random head produces large gradients flowing into a good backbone**, which is precisely the mechanism that destroys it.
- **Discriminative learning rates**: lower for early layers, higher for late ones, matching §1's finding about how much each layer should move.
- **Watch for catastrophic forgetting** if the model must retain its original capability — fine-tuning on a narrow task degrades performance on everything else, sometimes sharply.
- **And evaluate on the original task as well as the new one** when that matters. **Fine-tuning is lossy**, and the loss is invisible if you only measure the thing you tuned for.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Yosinski, Clune, Bengio & Lipson — How transferable are features in deep neural networks? (free)	The §1 measurement — transferability layer by layer, including the negative transfer result.
Radford et al. — Learning Transferable Visual Models From Natural Language Supervision (CLIP) (free)	The §2 objective with the broadest consequences. The zero-shot results section is the surprising part.
Hu et al. — LoRA: Low-Rank Adaptation of Large Language Models (free)	The §3 method, including the intrinsic-rank evidence that motivates it.
Raghu et al. — Transfusion: Understanding Transfer Learning for Medical Imaging (free)	The §4 caution, measured carefully. Required reading before claiming ImageNet pretraining helped.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Fine-tune a pretrained model** on your project and compare against training from scratch on the same split.
- 2 **Measure transferability by depth**: freeze the first k layers for k from 0 to all, and plot accuracy against k .

- 3 **Use too high a learning rate** when fine-tuning and demonstrate the destroyed features.
- 4 Add warmup and backbone freezing for the first epoch, and compare.
- 5 Implement a contrastive self-supervised objective and pretrain on unlabelled data.
- 6 **Vary the augmentations** in that objective and show the representation changes accordingly.
- 7 **Implement LoRA** and compare against full fine-tuning on quality, memory, and training time.
- 8 **Initialise B randomly instead of zero** and report what happens.
- 9 Merge the LoRA weights and verify inference is identical in cost to the base model.
- 10 **Find a domain pair where transfer hurts** and document it.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why do pretrained features transfer, and how does transferability vary with depth?
- 2 Compare five adaptation strategies and when each applies.
- 3 What is a pretext task, and why did self-supervision matter so much?
- 4 Give five pretraining objectives and their domains.
- 5 What did CLIP make possible that was not expected?
- 6 Explain LoRA's observation, its parameter saving, and its deployment advantage.
- 7 Why initialise B to zero?
- 8 Give three cases where transfer fails, and the baseline you must run.
- 9 Give five practices for fine-tuning without destroying the model.